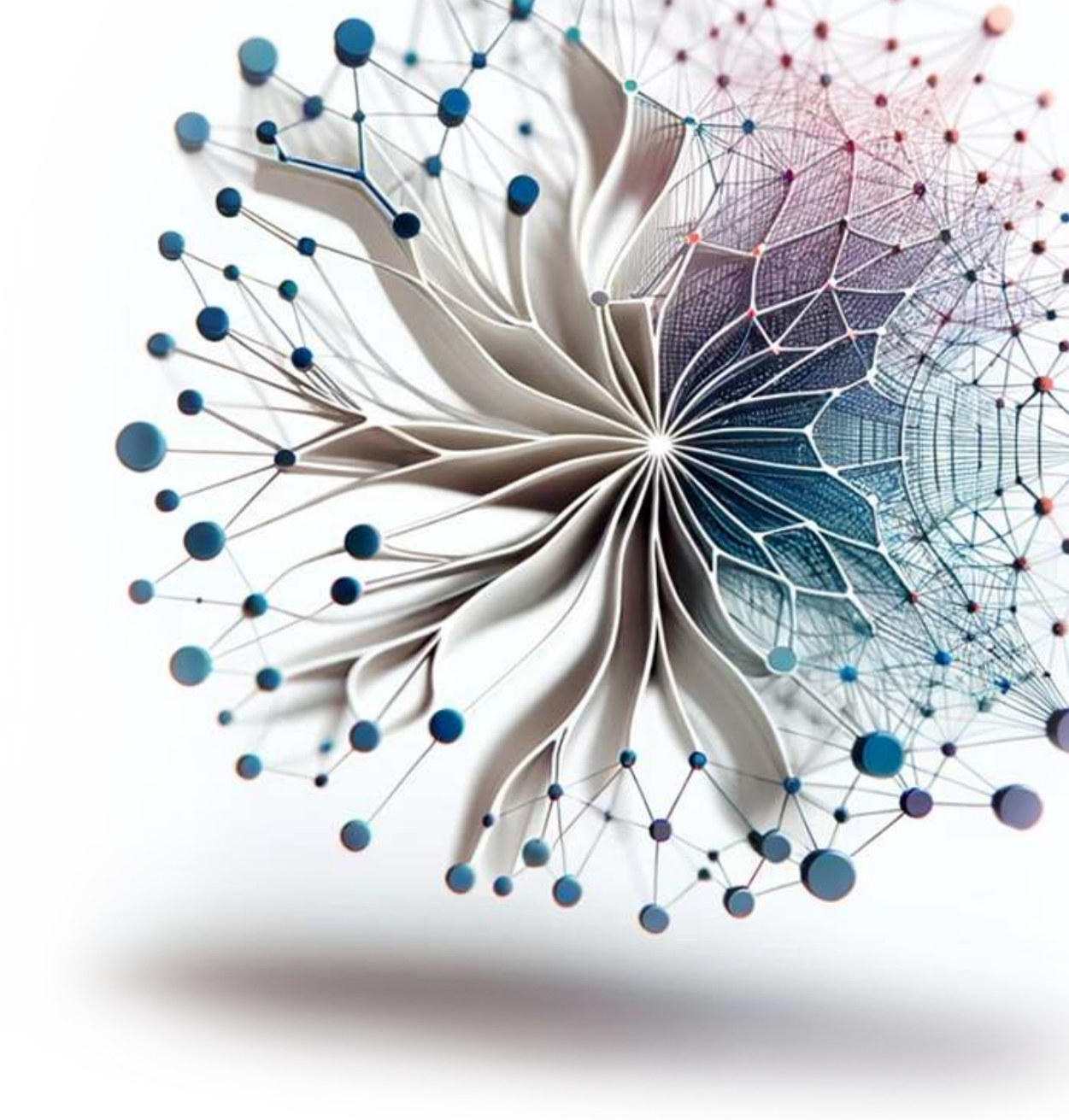

Introduction to

Machine Learning Systems

Vijay
Janapa Reddi

Introduction to Machine Learning Systems

Introduction to Machine Learning Systems

Vijay Janapa Reddi

The MIT Press
Cambridge, Massachusetts
London, England

The MIT Press
Massachusetts Institute of Technology
77 Massachusetts Avenue
Cambridge, MA 02139
mitpress.mit.edu

The MIT Press would like to thank the anonymous peer reviewers who provided comments on drafts of this book. The generous work of academic experts is essential for establishing the authority and quality of our publications. We acknowledge with gratitude the contributions of these otherwise uncredited readers. This book was prepared and formatted in Quarto by Željko Hrček. Printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data is available.

ISBN: 978-0-262-05888-9

EU Authorised Representative: Easy Access System Europe, Mustamäe tee 50, 10621 Tallinn, Estonia | Email: gpsr.requests@easproject.com

In loving memory of my father,
whose passion for learning and unwavering commitment to quality
challenge me daily to strive for excellence.

And to my mother,
who taught me, by instinct more than instruction,
that knowledge—like everything else worth having—
only matters if you give it away.

Table of Contents

Author's Note

The world is rushing to build AI systems. It is not yet engineering them. That gap is what we mean by AI engineering. Who designs the training infrastructure? Who builds serving systems that scale? Who optimizes models to run on a phone or a sensor? Who architects the accelerators those models execute on? That work is AI engineering: the discipline of building efficient, reliable, safe, and robust intelligent systems that operate in the real world, not models in isolation. Yet the work itself is still not recognized as a discipline.

The consequences are visible in practice. By most industry estimates, the vast majority of AI projects never reach production or fail to deliver the value they promised. The failures are not exotic: a model degrades silently because no one built monitoring for distribution shift; a system that worked in the lab fails at the edge because latency requirements were never communicated to the team selecting architectures; a deployment pipeline breaks because the team retrained a model without versioning the data and cannot reproduce last week's results. These are not research problems. They are engineering problems, and they recur because the field lacks the shared principles, vocabulary, and training that a discipline provides.

That missing discipline is also reflected in fragmentation. The knowledge to build ML systems exists, but it is scattered across silos. One course teaches compilers. Another covers hardware architecture. A third focuses on data pipelines. A fourth introduces the mathematics of optimization. Each is valuable on its own, but none of them teaches the *system*, and the result is engineers who understand individual components but not how they fit together. Nobody learns how a computer works by studying the processor in one course, the memory hierarchy in another, and the interconnect in a third, without ever building a complete machine. A computer is an integrated machine; understanding it means understanding where bottlenecks form between components, where trade-offs hide, and where one design decision constrains every other. An ML system is no different. Data pipelines, model architectures, software frameworks, and accelerator hardware constrain each other and fail together. The engineer who understands only one piece will build systems that break at the seams.

This book addresses that fragmentation by providing the missing integration: not a survey of tools, not a collection of recipes, but the enduring principles that govern ML systems regardless of which framework or accelerator is fashionable this year. We do not teach operating systems by starting with distributed operating systems or embedded operating systems; we teach the fundamentals (scheduling, memory management, concurrency) and then specialization follows. That philosophy guides this book. Whether you are building a model that runs on a microcontroller or training one across a thousand GPUs, the principles are the same: memory bandwidth limits data movement, arithmetic intensity governs compute utilization, and the interplay between data, models, and hardware determines what is practical to build. The scale changes; the principles do not.

The existence of such principles is precisely what makes a discipline possible. Software engineering and computer engineering each emerged when practitioners recognized that building reliable systems required its own body of knowledge, distinct from the science that produced the underlying ideas. AI engineering is at that same inflection point. The best textbooks I encountered as a student, in both traditions, did not teach me subjects; they taught me how to think. They took vast, messy landscapes and revealed the structure underneath. That is what I have tried to do here.

— Vijay Janapa Reddi

Notation

Machine Learning Systems spans **Machine Learning** (computer science/statistics) and **Systems** (computer architecture/hardware). Each field developed its notation independently, and many symbols mean different things depending on the community, paper, or literature using them. This collision creates real confusion when the disciplines merge. The conventions below establish a single notation for eliminating ambiguity.

Consider a simple statement: *"Increasing B improves throughput."* To an ML researcher, B means batch size. To a hardware engineer, B means bandwidth. Both interpretations are correct in their respective fields, but in ML Systems we need both concepts in the same equation—hence the need for a single, consistent convention.

The Iron Law of ML Systems

THE fundamental performance equation of this book, introduced in the opening chapter, is:

$$T = \frac{D_{\text{vol}}}{\text{BW}} + \frac{O}{R_{\text{peak}} \cdot \eta_{\text{hw}}} + L_{\text{lat}}$$

Each variable was chosen deliberately to avoid collision with standard ML terminology.

Symbol	Definition	Unit	Why This Symbol?
T	**Time**	seconds	Unambiguous. Wall-clock time for an operation.
D_{vol}	**Data Volume**	bytes	**Avoids collision with D (Dataset Size)**. In scaling laws, D means training tokens. Here we need bytes moved through memory. The subscript disambiguates.
BW	**Bandwidth**	bytes/s	**Avoids collision with B (Batch Size)**. Physics uses B for bandwidth, but every ML paper uses B for batch size. We preserve the ML convention.
O	**Operations**	FLOPs	Total floating-point operations. Clean in equations (vs. "Ops").
R_{peak}	**Peak Rate**	FLOP/s	**Avoids collision with P (Parameters)**. Roofline models use P for peak performance, but ML universally uses P for parameter count. We preserve the ML convention.
η_{hw}	**Efficiency**	—	Hardware utilization ($0 \leq \eta_{\text{hw}} \leq 1$). Avoids collision with learning rate (η).
L_{lat}	**Latency**	seconds	**Avoids collision with $\mathcal{L}$ (Loss)**. Fixed overhead time (kernel launch, network RTT). The subscript distinguishes from the loss function.

Why these choices matter

Without careful notation, sentences become ambiguous:

> *"Reducing D improves performance."*

The sentence has two possible readings:

- Reducing **dataset size** (fewer training samples) can speed training but may reduce accuracy.
- Reducing **data volume moved** (for example, through compression or quantization) can speed inference, with accuracy effects that depend on the technique and calibration.

With our notation, we can write precisely:

> *"Reducing D_{vol} through FP32-to-INT8 quantization cuts parameter memory traffic to one quarter while D (training data) remains unchanged."*

Our notation makes such ambiguity explicit: *"BW limits throughput"* is unambiguous.

Subscripted variants

When a multi-letter quantity name takes a descriptive subscript (for example, distinguishing storage bandwidth from network bandwidth), wrap both the root and the subscript in `\text{}`:

- BW_{disk}, $\text{BW}_{\text{network}}$, $\text{BW}_{\text{accelerator}}$
- D_{vol}, R_{peak}, L_{lat}, η_{hw}
- E_{move}, E_{compute}, E_{total}

Without `\text{}`, math mode renders each letter as an italic variable and the spacing collapses to look like a product, not a label.

The Degradation Equation

The silent failure mode of ML systems is captured by the degradation equation introduced in the opening chapter. Some symbols below (such as τ) appear in chapter prose rather than in the block equation itself; defining them centrally keeps the canonical form unambiguous when they do appear.

$$\text{Accuracy}(t) \approx \text{Accuracy}_0 - \lambda \cdot \mathcal{D}(P_t \| P_0)$$

Symbol	Definition	Unit/Type	Notes
$\text{Accuracy}(t)$	**Accuracy at Time t**	Scalar	Model accuracy after the model has been deployed for time t.
Accuracy_0	**Initial Accuracy**	Scalar	Model accuracy at deployment time.
λ	**Sensitivity**	Scalar	Model sensitivity to distribution shift. Architecture-dependent. *(Not wavelength.)*
P_t	**Current Distribution**	Distribution	The data distribution at time t. *(Not parameters—use P for parameter count.)*
P_0	**Training Distribution**	Distribution	The data distribution at training time.
$\mathcal{D}(P_t \| P_0)$	**Statistical Divergence**	Scalar ≥ 0	Measures how far P_t has drifted from P_0. Common choices: KL divergence, total variation, Wasserstein. *(Calligraphic to avoid collision with D = dataset size.)*
τ	**Drift Threshold**	Scalar > 0	Retraining is triggered when $\mathcal{D}(P_t \| P_0) > \tau$.

The Energy Corollary

The energy cost of ML workloads follows from the iron law of ML systems and decomposes as:

$$E_{\text{total}} \approx D_{\text{vol}} \times E_{\text{move}} + O \times E_{\text{compute}}$$

Symbol	Definition	Unit	Notes
E_{total}	**Total Energy**	joules	Total energy consumed by an ML workload, decomposed into data-movement and compute terms.
E_{move}	**Energy per Byte Moved**	joules/byte	Energy cost of data movement. Dominates total energy ($E_{\text{move}} \gg E_{\text{compute}}$).
E_{compute}	**Energy per Operation**	joules/FLOP	Energy cost of a single arithmetic operation.

Deep Learning Notation

We follow standard deep learning conventions (Goodfellow et al. 2016) with explicit disambiguation for systems variables.

Symbol	Definition	Dimensions/Type
B	**Batch Size**	Integer. The number of samples processed in parallel. *(Never bandwidth.)*
P	**Parameters**	Integer. The total count of trainable weights in a model. *(Never peak FLOP/s.)*
D	**Dataset Size**	Integer. Number of training samples or tokens. *(Never data volume in bytes—use D_{vol}.)*
S	**Sequence Length**	Integer. Number of tokens or time steps.
d	**Hidden Dimension**	Integer. Size of the hidden state vector.
N_L	**Number of Layers**	Integer. Total number of layers in a network.
ℓ	**Layer Index**	Integer. Index for a layer. Use instead of bare L when indexing layers, since L collides with loss and latency.
$\mathcal{L}$	**Loss Function**	Scalar. The objective function minimized during training.
η	**Learning Rate**	Scalar. Step size for the optimizer. *(Never bare for hardware efficiency—use η_{hw}.)*
θ	**Model Weights**	Vector/Matrix. The set of all learnable parameters.
$p(x)$	**Distribution**	Probability mass/density for a random variable. Use lowercase $p(\cdot)$ for generic distributions to avoid collision with P (parameter count).
$p(y \mid x)$	**Conditional Distribution**	Conditional probability/density. Use this form for generic label relationships; reserve P_0 and P_t for the degradation equation's training/current distributions.
$\Pr(E)$	**Event Probability**	Probability of an event E. Use for event statements such as $\Pr(\text{batch} = 0)$; use $p(x)$ and $p(y \mid x)$ for distributions.

Local matrix algebra may use A, B, and C for operands when the equation explicitly introduces matrices (for example, GEMM $C = \alpha AB + \beta C$). This is a local linear-algebra convention, not a global override of B as batch size.

Units and Precision

- Physical units: This book uses SI (metric) units throughout, including meters, kilograms, seconds, watts, and °C, consistent with standard engineering and scientific practice. Where source data was originally reported in imperial units, we convert to SI and note the original values parenthetically. A space always separates the number from the unit (for example, 100 ms, 2 TB/s).
- Data and memory: We use **decimal SI prefixes only**: KB = 10^3 bytes, MB = 10^6, GB = 10^9, TB = 10^{12}. We do not use binary-prefixed units in prose; all capacities, throughputs, and model sizes are reported in decimal units (for example, 80 GB, 2 TB/s, 102 MB).
- Compute: We distinguish operation counts from rates. Total work uses FLOPs (for example, GFLOPs, TFLOPs), while throughput uses FLOP/s with decimal prefixes (for example, GFLOP/s, TFLOP/s).
  - 1 TFLOP = 10^{12} FLOPs
  - 1 TFLOP/s = 10^{12} FLOPs per second
  - Arithmetic intensity conventionally uses FLOP/byte as a unit ratio (floating-point operations per byte moved). This is a ratio unit, not a total-work symbol or throughput symbol.
- Currency: Dollar amounts use the dollar sign (\$); unless otherwise noted, dollar-denominated costs are U.S. dollars (USD).
- Precision:
  - **FP64**: Double precision (8 bytes)
  - **FP32**: Single precision (4 bytes)

- **TF32**: TensorFloat-32 (19-bit compute format, commonly stored as FP32)
- **FP16**: Half precision (2 bytes, standard range)
- **BF16**: Brain float (2 bytes, wide dynamic range)
- **FP8**: Quarter precision (1 byte, E4M3 or E5M2 format)
- **FP4**: 4-bit floating-point format
- **INT8**: 8-bit integer (1 byte)
- **INT4**: 4-bit integer; lower integer precisions follow the same uppercase `INTn` pattern (for example, INT3, INT2)

Quick Reference: Resolving Collisions

Common collision points in ML Systems literature include:

Symbol	ML Meaning	Systems Meaning	Our Convention
B	Batch Size	Bandwidth	**Batch Size**. Use BW for bandwidth.
P	Parameters	Peak FLOP/s	**Parameters**. Use R_{peak} for peak rate.
D	Dataset Size	Data Volume	**Dataset Size**. Use D_{vol} for bytes moved.
L	Loss	Latency	**Loss** ($\mathcal{L}$). Use L_{lat} for latency.
η	Learning Rate	Efficiency	**Learning Rate**. Use η_{hw} for efficiency.

The general principle: **ML conventions take precedence for single letters**; systems concepts get subscripts or multi-letter symbols. This reflects the primary audience (ML practitioners learning systems) and preserves compatibility with the vast ML literature.

I

FOUNDATIONS OF ML SYSTEMS

Part I

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press.